

Word Length, Byte Storage, Endianness, and Tags

1. Word Length

Definition

- **Word length** refers to the number of bits a computer processor handles as a single unit of data.
- It defines the **amount of data the CPU can process at once** and determines the **size of registers, memory address range, and data bus width**.

Common Word Lengths

Word Length	Architecture	Max Addressable Memory
8-bit	Old Microcontrollers	256 bytes (2^8)
16-bit	Intel 8086, etc.	64 KB (2^{16})
32-bit	x86	4 GB (2^{32})
64-bit	x64, ARM64	18 EB (2^{64})

2. Byte and Byte Storage

Definition

- A **byte** is the **basic unit of data storage** in digital systems.
- 1 byte = 8 bits
- Bytes are used to represent characters (e.g., ASCII) and store all kinds of digital information.

Storage Units

Unit	Size in Bytes
1 Byte	8 bits
1 Kilobyte (KB)	1024 Bytes
1 Megabyte (MB)	1024 KB
1 Gigabyte (GB)	1024 MB
1 Terabyte (TB)	1024 GB

3. Big Endian and Little Endian

Endianness

Endianness refers to the **byte order** used to represent multi-byte data types (e.g., 16-bit, 32-bit, or 64-bit integers) in memory.

Big Endian

- Stores **most significant byte (MSB)** at the **lowest memory address**.
- Similar to how humans write numbers (left to right).

Example: To store 0x12345678:

- Address: 0x00 0x01 0x02 0x03
- Data: 0x12 0x34 0x56 0x78

Little Endian

- Stores **least significant byte (LSB)** at the **lowest memory address**.
- Used by most Intel processors.

Example: To store 0x12345678:

- Address: 0x00 0x01 0x02 0x03
- Data: 0x78 0x56 0x34 0x12

Endianness Comparison

Feature	Big Endian	Little Endian
MSB Location	Low Address	High Address
CPU Examples	PowerPC, SPARC	Intel, AMD
Human-Like	Yes	No

4. Tags

Definition

- A **Tag** is a group of **extra bits** associated with each **information word** to **identify the type of data** the word contains.
- This helps the system **interpret or process the word appropriately**.

Purpose

Technique to determine the type of a word by associating **tag bits** with each data word.

$$\text{Word} = \text{Tag Bits} + \text{Information Bits}$$

Example: In the **B6500/7500 Series**, words include a **tag field** and a **parity bit** for error checking.

Tagged-word Format (Example from B6500/7500 Series)

Bit Position	Field Name
0	Tag
1–50	Information/Data
51	Parity Bit

Advantages of Tags

- **Simplifies instruction specifications** by directly identifying operand types.
- **Improves reliability** by allowing **hardware-level checks** for **software errors**.
- Facilitates **dynamic type checking** during program execution.

Disadvantages of Tags

- **Increases memory size** due to extra bits per word.
- **Adds hardware cost** without increasing raw computational power.
- Requires changes in architecture to support tagged memory access and processing.

Applications of Tags

- **Tagged Architectures** (e.g., Burroughs B6500/7500 systems)
- **Cache Memory Systems** – tags identify blocks in associative memory.
- **Dynamic Languages** (e.g., Lisp, Python) – runtime type checking.

- **Error Detection/Correction** – parity/tag bits used.

Summary

Feature	Description
Tag Bits	Identify data type or purpose
Used In	Tagged memory, cache systems, dynamic languages
Pros	Simplicity, type safety, error detection
Cons	Memory and hardware overhead